

Concepto general

Cada empresa tiene su propio formulario web (HTML) con su diseño, colores y campos únicos. Todos comparten un único archivo JavaScript — **flux-reservas-core.js** — que contiene la lógica de conexión con Firebase, lectura de horarios y envío de reservas. El HTML solo declara sus campos y vincula su empresa con una sola línea.

Arquitectura: 1 script, N formularios

Archivo	Quién lo toca	Contenido
flux-reservas-core.js	Nunca se modifica	Toda la lógica: Firebase, horarios, aforo, bloqueos, envío a Firestore
restaurante-a.html	Diseñador del cliente A	CSS propio · Campos propios · 1 línea con UID
peluqueria-b.html	Diseñador del cliente B	CSS propio · Campos propios · 1 línea con UID
hotel-c.html	Diseñador del cliente C	CSS propio · Campos propios · 1 línea con UID

Todos los HTML se hospedan en la misma carpeta del servidor junto con **flux-reservas-core.js**. Solo ese archivo incluye credenciales de Firebase.

Estructura mínima de cada HTML

Estos son los 4 bloques imprescindibles. El resto (CSS, estructura, campos) es completamente libre:

Bloque 1 — Firebase SDK (siempre igual, 3 líneas)

Incluir al inicio del **<body>** o al final. Son las mismas para todas las empresas:

```
<script src="https://www.gstatic.com/firebasejs/9.23.0/firebase-app-compat.js"></script>
<script src="https://www.gstatic.com/firebasejs/9.23.0/firebase-auth-compat.js"></script>
<script src="https://www.gstatic.com/firebasejs/9.23.0/firebase-firestore-compat.js"></script>
```

Bloque 2 — Script compartido de lógica

Solo una línea. Misma para todos:

```
<script src="flux-reservas-core.js"></script>
```

Bloque 3 — Formulario HTML con campos propios

El **id** del **<form>** es **reservaForm** por defecto (configurable). Los campos **fecha** y **hora** son obligatorios. El resto son libres:

```
<form id="reservaForm">
<input type="date" id="fecha" required> <!-- OBLIGATORIO -->
<select id="hora" required></select> <!-- OBLIGATORIO: el script lo rellena -->
<span id="slotInfo"></span> <!-- muestra disponibilidad -->
<input type="text" id="nombre" required> <!-- campo propio -->
<input type="tel" id="telefono"> <!-- campo propio -->
<!-- ... cualquier otro campo que necesite la empresa -->
<button type="submit" id="btnReservar">Reservar</button>
<div id="formMsg"></div>
</form>
```

Bloque 4 — Vincular con la empresa (1 línea a cambiar)

Al final del <body>. La única línea diferente entre empresas es **EMPRESA_ID**:

```
<script>
var EMPRESA_ID = 'TUz8G0nQ60X8ejiov7c5GM9LFP12'; // ← SOLO ESTO CAMBIA
FluixReservas.init(EMPRESA_ID, {
// Campos extra a guardar en Firestore (ids de tus inputs)
camposExtra: ['nombre', 'telefono', 'email', 'personas']
});
</script>
```

Dónde encontrar el EMPRESA_ID

Paso	Acción
1	Abrir la app Fluix con la cuenta del negocio
2	Ir a Módulo de Reservas → Configuración de Reservas
3	Pulsar la pestaña 'Web' (la cuarta)
4	El ID aparece en el panel 'Tu URL de reservas' — pulsar Copiar

Opciones completas de FluixReservas.init()

Parámetro	Valor por defecto	Para qué sirve
formId	'reservaForm'	id del <form> principal
campoFecha	'fecha'	id del input type=date
campoHora	'hora'	id del select de horario (rellena automáticamente)
idSlotInfo	'slotInfo'	id del elemento que muestra disponibilidad
idBotonEnviar	'btnReservar'	id del botón submit
idMensaje	'formMsg'	id del div de resultado (éxito/error)
idAvisoInactivo	'reservaDesactivadoAviso'	id del div que se muestra si web desactivada
camposExtra	[]	ids de los inputs a guardar en Firestore
mensajeExito	texto genérico	function(datos) → string de mensaje de éxito
onExito	undefined	function(datos) → callback tras reserva guardada

Nota: Si usas los id por defecto (reservaForm, fecha, hora, slotInfo, btnReservar, formMsg) no necesitas declarar ninguna opción salvo **camposExtra**.

Cómo declarar camposExtra

El array **camposExtra** contiene los ids de los inputs que quieres guardar en Firestore junto con la reserva. El script detecta automáticamente el tipo de cada campo:

Tipo de campo	Cómo se recoge el valor
input text/email/tel/number	element.value.trim()
select	element.value
textarea	element.value.trim()
input type=radio	Se busca el radio con :checked del mismo name

Tipo de campo	Cómo se recoge el valor
input type=checkbox	Devuelve 'sí' si está marcado, 'no' si no

Ejemplo con 7 campos (restaurante con campo exclusivo):

```
camposExtra: ['nombre', 'telefono', 'email', 'personas', 'zona', 'alergenos', 'ocasion']
```

Ejemplo con 4 campos (peluquería):

```
camposExtra: ['nombre', 'telefono', 'servicio', 'notas']
```

Qué lee el formulario de Firestore automáticamente

Todo esto se configura en la app Fluix y se sincroniza sin tocar el HTML:

Qué se configura en la app	Efecto en el formulario web
Días activos de la semana	Bloquea días no activos al seleccionar fecha
Horario de apertura/cierre por día	Genera los slots de hora automáticamente
Slots personalizados por día	Muestra exactamente esos horarios (no auto-generados)
Duración del slot (15/30/45/60 min...)	Determina la frecuencia de los slots
Días específicos cerrados + motivo	Bloquea esa fecha y muestra el motivo
Días recurrentes cerrados (ej: lunes)	Bloquea todos los lunes, martes, etc.
Intervalo de fechas (vacaciones)	Bloquea ese rango y muestra el motivo
Aforo máximo por franja	Deshabilita slots que ya tienen ese número de reservas
Reservas web activas (toggle)	Oculto el formulario si está desactivado

La sincronización es en **tiempo real**: si el propietario cambia algo en la app y guarda, el formulario web lo refleja al instante (Firestore onSnapshot).

Dónde se guardan las reservas en Firestore

Cada reserva enviada desde el formulario web se guarda en:

```
empresas/{EMPRESA_ID}/reservas/{nuevaReservaId}
```

Con los siguientes campos:

Campo	Valor
fecha_hora	Timestamp Firestore (fecha + hora seleccionada)
hora	String 'HH:mm' (ej: '14:30')
estado	'PENDIENTE'
origen	'web'
fecha_creacion	Timestamp servidor
...camposExtra	Todos los campos declarados en camposExtra con su valor

La reserva aparece inmediatamente en el módulo de Reservas de la app del negocio.

Checklist de elementos HTML obligatorios

- **input type=date id="fecha"** — selector de fecha (el script le pone el mínimo = hoy)
- **select id="hora"** — el script lo rellena con los slots disponibles
- **button type=submit id="btnReservar"** — envía el formulario

- `span id="slotInfo"` — (recomendado) muestra el mensaje de disponibilidad
- `div id="formMsg"` — (recomendado) muestra el resultado del envío
- `p id="reservaDesactivadoAviso"` — (recomendado) aviso cuando reservas desactivadas

Clases CSS que añade el script (para estilizar)

El script añade estas clases al elemento `#slotInfo`. Puedes definir su estilo:

Clase CSS	Cuándo aparece
<code>.flux-slot-info</code>	Siempre (clase base)
<code>.flux-slot-info.flux-ok</code>	Slot disponible
<code>.flux-slot-info.flux-full</code>	Slot completo
<code>.flux-slot-info.flux-closed</code>	Fecha bloqueada

El mensaje de resultado (`#formMsg`) usa las clases estándar de Flux:

```
.form-msg.success { color: verde; background: verde translúcido }
.form-msg.error { color: rojo; background: rojo translúcido }
```

Cosas importantes a tener en cuenta

1. El script usa autenticación anónima de Firebase

Para poder leer/escribir en Firestore, el script hace login anónimo automáticamente. No requiere que el cliente esté registrado. Asegúrate de que la autenticación anónima esté habilitada en Firebase Console → Authentication → Sign-in providers.

2. Reglas de Firestore

Las reglas de Firestore deben permitir a usuarios anónimos:

- LEER: `empresas/{empresald}/configuracion/reservas_web`
- LEER: `empresas/{empresald}/reservas` (para contar el aforo por fecha)
- ESCRIBIR: `empresas/{empresald}/reservas` (para crear nuevas reservas)

3. El select de hora es rellenado por el script — no pongas opciones

El `<select id="hora">` debe estar vacío o con una opción placeholder. El script lo llena automáticamente cuando el usuario selecciona una fecha. Si pones opciones manuales, el script las reemplazará.

4. El formulario detecta si las reservas están desactivadas

Si el propietario desactiva las reservas web desde la app, el formulario se oculta automáticamente y aparece el elemento `#reservaDesactivadoAviso`. Pon ese elemento en el HTML aunque sea con `display:none` — el script lo gestiona.

5. El aforo se consulta en Firestore (no localStorage)

A diferencia de versiones anteriores, el aforo máximo por franja se consulta directamente en Firestore al seleccionar una fecha. Esto garantiza que si dos personas abren el formulario a la vez, ambas ven la disponibilidad real.

Ejemplo mínimo completo (5 campos)

```
<!-- Firebase SDK -->
<script src="firebase-app-compat.js"></script>
<script src="firebase-auth-compat.js"></script>
<script src="firebase-firestore-compat.js"></script>
<script src="flux-reservas-core.js"></script>
```

```

<form id='reservaForm'>
<input type='text' id='nombre' required>
<input type='tel' id='telefono' required>
<input type='date' id='fecha' required>
<select id='hora' required></select>
<span id='slotInfo'></span>
<select id='personas'>...</select>
<button type='submit' id='btnReservar'>Reservar</button>
<div id='formMsg'></div>
</form>

<script>
FluixReservas.init('EMPRESA_ID_AQUI', {
camposExtra: ['nombre', 'telefono', 'personas']
});
</script>

```

Resumen: lo que cambia por empresa

Qué	¿Cambia entre empresas?	Notas
flux-reservas-core.js	NO	Un único archivo para todos
Firebase SDK (3 scripts)	NO	Siempre los mismos CDN
EMPRESA_ID	SÍ — única diferencia	Copiar desde la app: Reservas → Web
camposExtra	SÍ	Los ids de los inputs de ese formulario
CSS / diseño visual	SÍ	Completamente libre
Campos del formulario	SÍ	Cada empresa pone los que necesita
mensajeExito (opcional)	SÍ	Para personalizar el texto de confirmación